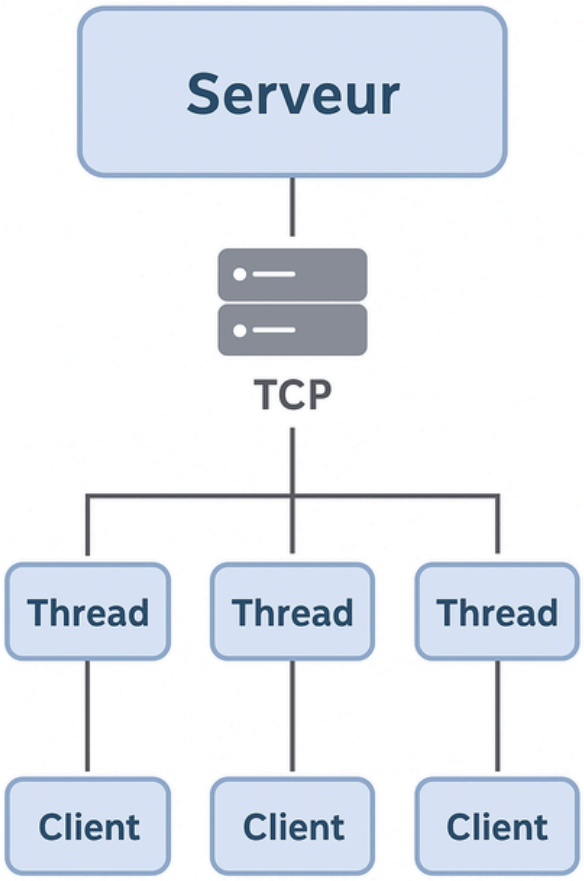
COMPTE RENDU TP3 A3.1

EYA CHIKHAOUI

3e Licence Sciences de l'Informatique (LSI 3 – 1.2)

1. Introduction :

- Dans le cadre du TP3 de la matière Développement d'Applications Réparties (DAR), cette première activité consiste à mettre en œuvre un serveur TCP capable d'accepter plusieurs connexions clientes.
- L'objectif est de compléter les connaissances acquises sur la communication client-serveur à travers les sockets, en introduisant le mécanisme du multithreading. Celui-ci permet au serveur de créer un thread pour traiter chaque client connecté, au lieu de gérer toutes les connexions dans un seul traitement séquentiel.
- L'application développée repose sur le langage Java et les classes de communication réseau disponibles dans la bibliothèque java.net, ainsi que sur les mécanismes de gestion des threads.



2. Objectifs de l'activité

Les principaux objectifs de cette activité sont :

- Mettre en place une communication basée sur le protocole TCP.
- Créer un serveur utilisant la classe ServerSocket.
- Accepter les connexions des clients à l'aide de la méthode accept().
- Identifier chaque client connecté à partir de son adresse distante.
- Attribuer un numéro d'ordre à chaque client.
- Utiliser les threads pour traiter les connexions clientes indépendamment.
- Envoyer à chaque client un message contenant son numéro d'ordre.



figure 1 : principe général

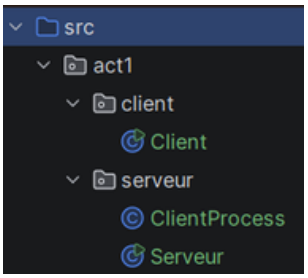


figure 2 : arborescence du projet

3. Environnement de développement

La réalisation de cette activité nécessite les éléments suivants :

- Langage de programmation : Java.
- Environnement de développement : IDE Java utilisé pour le projet.
- Technologie de communication : sockets TCP.

4. Organisation du projet

Le projet est organisé de manière à séparer les composants du serveur et du client. Pour l'activité 3-1, le package act1 contient les éléments nécessaires à la communication.

Les trois classes principales utilisées sont :

- Serveur :
  - responsable de l'ouverture du serveur et de l'acceptation des connexions.
- ClientProcess :
  - responsable du traitement individuel de chaque client dans un thread.
- Client :
  - permet d'établir une connexion avec le serveur et de recevoir le message envoyé.

5. Côté Serveur :

5.1. Classe Serveur.java

La classe Serveur constitue le point d'entrée de l'application côté serveur. Elle permet d'ouvrir une socket serveur sur le port 1234 et d'attendre les connexions des clients.
La création de l'objet ServerSocket permet au serveur d'écouter les demandes de connexion entrantes. La méthode accept() bloque l'exécution jusqu'à ce qu'un client établisse une connexion.

Après chaque connexion acceptée, le serveur :

1. Incrémente le compteur de clients connectés.
2. Attribue un numéro d'ordre au nouveau client.
3. Récupère son adresse distante avec getRemoteSocketAddress().
4. Crée un nouvel objet ClientProcess.
5. Lance un thread avec la méthode start().

Cette organisation permet au serveur de revenir à l'attente d'une nouvelle connexion pendant que le thread créé prend en charge le client déjà accepté.

Serveur.java

```java
package act1.serveur;
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
public class Serveur {  new *
    private static final int PORT = 1234;  2 usages
    public static void main(String[] args) {  new *
        int numeroClient = 0;

        try (ServerSocket serveurSocket = new ServerSocket(PORT)) {
            System.out.println("Serveur démarré...");
            System.out.println("En attente des connexions sur le port " + PORT);

            while (true) {
                // Attendre et accepter une connexion cliente
                Socket socketClient = serveurSocket.accept();
                // Attribuer un numéro au client
                numeroClient++;
                int numero = numeroClient;
                // Afficher les informations du client
                System.out.println("Client n°" + numero + " connecté depuis : " + socketClient.getRemoteSocketAddress());
                // Créer un thread pour traiter ce client
                Thread thread = new Thread(new ClientProcess(socketClient, numero));
                // Démarrer le thread
                thread.start();
            }

        } catch (IOException e) {
            System.out.println("Erreur du serveur : " + e.getMessage());
        }
    }
}
```

Figure 3 – Implémentation de la classe Serveur.java

5.2. Classe ClientProcess.java

La classe ClientProcess représente le traitement associé à un client connecté.
Elle implémente l'interface Runnable, ce qui permet d'exécuter son traitement dans un thread indépendant.

Son constructeur reçoit deux informations :

- La socket représentant la connexion avec le client.
- Le numéro d'ordre attribué par le serveur.

Dans la méthode run(), la classe crée un flux de sortie à l'aide de PrintWriter. Elle utilise ensuite println() pour transmettre au client un message contenant son numéro d'ordre.
L'option true passée au constructeur de PrintWriter active l'envoi automatique du contenu après les opérations d'écriture terminées par une nouvelle ligne.
Enfin, la socket cliente est fermée après le traitement.

ClientProcess.java

```java
package act1.serveur;
import java.io.IOException;
import java.io.PrintWriter;
import java.net.Socket;
public class ClientProcess implements Runnable {  1 usage  new *
    private Socket socketClient;  3 usages
    private int numeroClient;  4 usages
    public ClientProcess(Socket socketClient, int numeroClient) {  1 usage  new *
        this.socketClient = socketClient;
        this.numeroClient = numeroClient;
    }
    @Override  new *
    public void run() {
        try {
            // Créer un flux de sortie vers le client
            PrintWriter sortie = new PrintWriter(socketClient.getOutputStream(),  autoFlush: true);
            // Envoyer le numéro d'ordre au client
            sortie.println("Vous êtes le client n°" + numeroClient);

            System.out.println("Message envoyé au client n°" + numeroClient);
        } catch (IOException e) {
            System.out.println("Erreur avec le client n°" + numeroClient + " : " + e.getMessage());
        } finally {
            try {
                socketClient.close();
            } catch (IOException e) {
                System.out.println("Erreur lors de la fermeture de la connexion.");
            }
        }
    }
}
```

Figure 4 – Code de la classe ClientProcess.java

6. Côté Client :

La classe Client permet de tester la communication avec le serveur.

Elle établit une connexion TCP vers le serveur local à l'aide de l'instruction :

new Socket("localhost", 1234)

Une fois la connexion établie, le client récupère le flux d'entrée de la socket. Les classes InputStreamReader et BufferedReader permettent de lire le message textuel transmis par le serveur.

La méthode readLine() récupère la ligne envoyée par le serveur, puis le client affiche le message reçu dans la console.

Client.java

```java
package act1.client;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
import java.net.Socket;

public class Client {  new *

    public static void main(String[] args) {  new *

        try (Socket socket = new Socket( host: "localhost",  port: 1234)) {

            System.out.println("Connexion au serveur réussie.");

            // Recevoir le message envoyé par le serveur
            BufferedReader entree = new BufferedReader(new InputStreamReader(socket.getInputStream())
            );

            String message = entree.readLine();

            System.out.println("Message du serveur : " + message);

        } catch (IOException e) {

            System.out.println("Erreur de connexion : " + e.getMessage());
        }
    }
}
```

Figure 5 – Code de la classe Client.java

Première vérification de la communication client-serveur

La vérification consiste à démarrer le serveur, puis à exécuter le programme client.

Le serveur doit rester en attente des connexions sur le port 1234. Lorsque le client se connecte, le serveur accepte la connexion, lui attribue un numéro d'ordre et lance le traitement correspondant.

Le client reçoit ensuite le message transmis par le serveur et l'affiche dans sa console.



Figure 6 – Console du serveur démarré

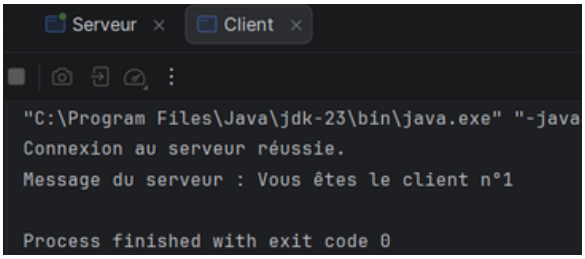


Figure 7 – Console serveur et console client après la connexion

Observation expérimentale :
à compléter après l'exécution réelle du programme, en indiquant le message affiché côté serveur et le message reçu côté client.

Tester plusieurs clients

Afin de vérifier la capacité du serveur à gérer plusieurs connexions clientes, plusieurs instances du programme client sont exécutées successivement pendant que le serveur reste actif.

À chaque nouvelle connexion, le serveur accepte la socket cliente, incrémente le compteur et attribue un numéro d'ordre différent. Il crée ensuite un thread chargé de traiter cette connexion et de transmettre le numéro correspondant au client.

Cette vérification permet d'observer le fonctionnement de la boucle d'acceptation du serveur et la création d'un traitement indépendant pour chaque client.

```
Serveur démarré...
En attente des connexions sur le port 1234
Client n°1 connecté depuis : /127.0.0.1:53983
Message envoyé au client n°1
Client n°2 connecté depuis : /127.0.0.1:25425
Message envoyé au client n°2
Client n°3 connecté depuis : /127.0.0.1:22792
Message envoyé au client n°3
```

Figure 8 – Acceptation successive de plusieurs connexions clientes par le serveur TCP

```
Connexion au serveur réussie.
Message du serveur : Vous êtes le client n°2
```

```
Connexion au serveur réussie.
Message du serveur : Vous êtes le client n°3
```

Figure 9 – Résultats affichés par les clients

Analyse des résultats :
à compléter après le test réel. Vérifier que chaque client reçoit son propre numéro et que le serveur continue d'accepter de nouvelles connexions.

lien de tp dans github:
https://github.com/eyaChikhaoui/TP3-DAR2026.git

Conclusion :
À travers cette activité, nous avons mis en pratique la communication client-serveur basée sur les sockets TCP et découvert le fonctionnement d'un serveur multithread en Java. La création d'un thread pour chaque connexion permet au serveur de traiter plusieurs clients indépendamment. Cette réalisation constitue une base importante pour la suite du TP, qui consiste à enrichir la communication client-serveur par l'échange d'objets et le partage de données entre plusieurs traitements.

Je reste disponible pour toute remarque ou suggestion d'amélioration.
Merci pour votre attention.